

```

/* Ejemplo 10.26 p. 380 */
/* ordenar alfabeticamente una lista de cadenas de caracteres,
   utilizando una formacion de punteros */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void reordenar(int n, char *x[]);

main() {
    int i, n = 0;
    char *x[0];

    printf("Introducir debajo cada cadena en una linea\n\n");
    printf("Escribir \'FIN\' para terminar\n\n");

    /* leer la lista de cadenas de caracteres */
    do {
        /* reservar memoria */
        x[n] = (char *) malloc(12 * sizeof(char));

        printf("cadena %d: ", n + 1);
        scanf("%s", x[n]);
    }
    while (strcmp(x[n++], "FIN"));

    /* escribir la lista reordenada de cadenas de caracteres */
    printf("\n\nLista reordenada de cadenas:\n");
    for (i = 0; i < n; ++i)
        printf("\ncadena %d: %s", i + 1, x[i]);
}

void reordenar(int n, char *x[]) /* reordenar la lista de cadenas */
{
    char *temp;
    int i, elem;

    for (elem = 0; elem < n - 1; ++elem)
        /* encontrar la menor de las cadenas restantes */
        for (i = elem + 1; i < n; ++i)

            if (strcmp(x[elem], x[i] > 0)) {
                /* intercambiar las dos cadenas */
                temp = x[elem];
                x[elem] = x[i];
                x[i] = temp;
            }
}

```

```

    }
    return;
}

```

```

/* Ejemplo 10.27 p. 382 */
static char *nombres[10] = {
    "PACIFICO",
    "ATLANTICO",
    "INDICO",
    "CARIBE",
    "BERING",
    "NEGRO",
    "ROJO",
    "NORTE",
    "BALITICO",
    "CASPIO"
};

```

```

/* Ejemplo 10.28 p. 383 */
/* convertir una fecha numérica (mm dd aaaa) en "día de la semana,
   mes, día, año"

   (por ejemplo: 5 24 1997 -> "Sábado, Mayo 24, 1997") */

#include <stdio.h>
void leerentrada(int *pm, int *pd, int *pa); /* prototipo de función */
int convertir(int mm, int dd, int aa); /* prototipo de función */

main()
{
    int mm, dd, aa;
    int dia_semana; /* día de la semana (0 -> Domingo,
                                     1 -> Lunes,
                                     ...
                                     6 -> Sábado) */

    static char *diasemana[] = {"Domingo", "Lunes", "Martes",
                                "Miércoles", "Jueves", "Viernes",
                                "Sábado"};

    static char *mes[] = {"Enero", "Febrero", "Marzo", "Abril", "Mayo",
                           "Junio", "Julio", "Agosto", "Septiembre",
                           "Octubre", "Noviembre", "Diciembre"};

    /* mensaje de entrada */
    printf ("Rutina de conversión de fecha\n Para PARAR, introducir 0 0 0");
}

```

```

leerentrada (&mm, &dd, &aa);

/* convertir fecha a día numérico de la semana */
while (mm > 0) {
    dia_semana = convertir (mm, dd, aa);
    printf("\n%s, %s %d, %d", diasemana[dia_semana], mes[mm-1], dd, aa);
    leerentrada(&mm, &dd, &aa);
}
}

void leerentrada (int *pm, int *pd, int *pa) /* leer la fecha numérica */
{
    printf("\n\nIntroducir mm dd aaaa: ");
    scanf("%d %d %d", pm, pd, pa);
    return;
}

int convertir(int mm, int dd, int aa) /* convertir una fecha en el
                                     día de la semana */
{
    long ndias;      /* número de días desde el comienzo de 1900 */
    long nciclos;    /* número de ciclos de 4 años después de 1900 */
    int nanios;      /* número de años después del último ciclo de 4 años */
    int dia;         /* día de la semana (0, 1, 2, 3, 4, 5 o 6) */

    /* conversiones numéricas */
    aa -= 1900;
    ndias = (long) (30.42 * (mm - 1)) + dd; /* día aproximado del año */
    if (mm == 2) ++ndias                    /* ajuste para febrero */
    if ((mm > 2) && (mm < 8)) --ndias;        /* ajuste para marzo-ju-
                                             lio */
    if ((aa % 4 == 0) && (mm > 2)) ++ndias; /* ajuste para el año bi-
                                             siesto */

    nciclos = aa / 4;                        /* ciclos de 4 años des-
                                             pués de 1900 */
    ndias += nciclos * 1461;                 /* añadir días por ciclos
                                             de 4 años */

    nanios = aa % 4;                        /* años después del último ciclo de 4 años
                                             */
    if (nanios > 0)                          /* añadir días por años después del último
                                             ciclo */
        ndias += 365 * nanios + 1;

    if (ndias > 59) --ndias; /* ajustar para 1900 (NO año bisiesto) */

    dia = ndias % 7;

```

```

    return(dia) ;
}

```

```

/* ejemplo 10.29 p. 389 */

```

```

int procesar(int (*) (int, int)); /* prototipo de función (anfitriona) */
int func1(int, int); /* prototipo de función (huésped) */
int func2(int, int); /* prototipo de función (huésped) */

```

```

main()
{
    int i, j;
    i = procesar (func1) ; /* se pasa func1 a procesar; devuelve un va-
                           lor para i */
    j = procesar (func2); /* se pasa func2 a procesar; devuelve un va-
                           lor para j */
}

```

```

procesar(int (*pf) (int, int)) /* definición de función anfitriona */
                                /* (el argumento formal es un puntero
                                a una función) */
{
    c = (*pf) (a, b); /* acceso a la función pasada a esta función;
                       devuelve un valor para c */
    return(c);
}

```

```

func1(int a, int b) /* definición de función huésped */
{
    int c;
    c =                /* utiliza a y b para evaluar c */
    return(c);
}

```

```

func2(int x, int y) /* definición de función huésped */
{
    int z;
    z =                /* utiliza x e y para evaluar z */
    return (z);
}

```

```

/* cálculos financieros personales */

```

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <math.h>

```

```

/* prototipos de funciones */
void tabla(double (*pf) (double i, int m, double n), double a, int m,
           double n);
double md1(double i, int m, double n);
double md2(double i, int m, double n);
double md3(double i, int m, double n);

main() /* calcular el valor futuro de una serie de depósitos men-
       suales */
{
    int m;      /* número de períodos de composición por año */
    double n;   /* número de años */
    double a;   /* cantidad de cada pago mensual */
    char frec;  /* indicador de frecuencia de composición */

    /* entrada de datos */
    printf("\nVALOR FUTURO DE UNA SERIE DE DEPOSITOS MENSUALES\n\n");
    printf("Cantidad de cada pago mensual: ");
    scanf("%lf", &a);
    printf("Número de años: ");
    scanf("%lf", &n);

    /* introducir frecuencia de composición */
    do {
        printf ("Frecuencia de composición (A, S, T, M, D, C): ");
        scanf("%1s", &frec),
        frec = toupper (frec);    /* convertir a mayúsculas */
        if (frec == 'A') {
            m = 1;
            printf("\nComposición anual \n");
        }
        else if (frec == 'S') {
            m = 2;
            printf("\nComposición semestral \n");
        }
        else if (frec == 'T') {
            m = 4;
            printf("\nComposición trimestral \n");
        }
        else if (frec == 'M') {
            m = 12;
            printf("\nComposición mensual \n");
        }
        else if (frec == 'D') {
            m = 360;
            printf("\nComposición diaria \n ");
        }
        else if (frec == 'C') {

```

```

        m = 0;
        printf("\nComposición continua\n" );
    }
    else
        printf("\nERROR - Por favor repita\n\n");
} while (frec != 'A' && frec != 'S' && frec != 'T' &&
        frec != 'M' && frec != 'D' && frec != 'C');

/* realizar los cálculos */
if (frec == 'C')
    tabla (md3 , a, m, n); /* composición continua */
else if (frec 'D')
    tabla (md2, a, m, n); /* composición diaria */
else
    tabla(md1, a, m, n); /* composición anual, semestral, tri-
                        mestral o mensual */
}

void tabla (double (*pf) (double i, int m, double n), double a, int m,
           double n)
/* generador de la tabla (esta función acepta un puntero a otra
función como argumento)
NOTA: double (*pf) (double i, int m, double n) es un PUNTERO A
FUNCION */
{
    int cont; /* contador del bucle */
    double i; /* tasa de interés anual */
    double f; /* valor futuro */

    printf("\nTasa de interés    Cantidad futura\n\n");
    for (cont = 1; cont <= 20; ++cont) {
        i = 0.01 * cont;
        f = a * (*pf) (i, m, n); /* EL ACCESO A LA FUNCION SE PASA
                                COMO UN PUNTERO */
        printf("        %2d        %.2f\n", cont, f);
    }
    return;
}

double md1(double i, int m, double n)
/* depósitos mensuales, composición periódica */
{
    double factor, razon;
    factor = 1 + i/m;
    razon = 12 * (pow(factor, m*n) - 1) / i;
    return(razon) ;
}

double md2(double i, int m, double n)

```

```

/* depósitos mensuales, composición diaria */
{
    double factor, razon;
    factor = 1 + i/m;
    razon = (pow(factor, m*n) - 1) / (pow(factor, m/12) - 1);
    return(razon);
}

```

```

double md3(double i, int nulo, double n)
/* depósitos mensuales, composición continua */
{
    double razon;
    razon = (exp(i*n) - 1) / (exp(i/12) - 1);
    return(razon);
}

```

```

/* ejemplo 10.31 p. 398 */

```

```

int *p;           /* p es un puntero a una cantidad entera */
int p[10];        /* p es un array de 10 punteros a ente-
                  ros */
int (*p)[10];     /* p es un puntero a un array de 10 ente-
                  ros */
int *p(void);     /* p es una función que devuelve un puntero
                  a entero */
int p(char *a);   /* p es una función que acepta un argumento
                  que es un puntero a carácter y devuelve
                  un entero */
int *p(char *a);  /* p es una función que acepta un argu-
                  mento que es un puntero a carácter y de-
                  vuelve un puntero a un entero */
int (*p)(char *a); /* p es un puntero a una función que acepta
                  un argumento que es un puntero a carác-
                  ter y devuelve un entero */

int (*p(char *a))[10]; /* p es una función que acepta un argu-
                  mento que es un puntero a carácter y de-
                  vuelve un puntero a un array de diez
                  enteros */
int p(char (*a)[]);   /* p es una función que acepta un argumento
                  que es un puntero a un array de caracte-
                  res y devuelve un entero */
int p(char *a[]);     /* p es una función que acepta un argumento
                  que es un array de punteros a caracteres
                  y devuelve un entero */
int *p(char a[]);     /* p es una función que acepta un argumento
                  que es un array de caracteres y devuelve
                  un puntero a entero */

```

```

int *p(char (*a)[]);    /* p es una función que acepta un argumento
                        que es un puntero a un array de caracte-
                        res y devuelve un puntero a entero */

int *p(char *a[]);      /* p es una función que acepta un argu-
                        mento que es un array de punteros a
                        caracteres y devuelve un puntero a ente-
                        ro */

int (*p)(char (*a)[]);  /* p es un puntero a una función que acepta
                        un argumento que es un puntero a un array
                        de caracteres y devuelve un entero */

int *(*p)(char (*a)[]); /* p es un puntero a una función que acepta
                        un argumento que es un puntero a un array
                        de caracteres y devuelve un puntero a
                        entero */

int *(*p)(char *a[]);   /* p es un puntero a una función que acepta
                        un argumento que es un array de punteros
                        a caracteres y devuelve un puntero a en-
                        tero */

int (*p[10]) void;      /* p es un array de 10 punteros a función;
                        cada función devuelve un entero */

int (*p[10])(char a);   /* p es un array de 10 punteros a función;
                        cada función acepta un argumento que es un
                        carácter y devuelve un entero */

int *(*p[10])(char a);  /* p es un array de 10 punteros a función;
                        cada función acepta un argumento que es
                        un carácter y devuelve un puntero a en-
                        tero */

int *(*p[10])(char *a); /* p es un array de 10 punteros a función;
                        cada función acepta un argumento que es
                        un puntero a carácter y devuelve un pun-
                        tero a entero */

```